김동현

실서비스 기준으로 문제를 해결하고, 변화에 유연하게 대응하는 구조를 설계하는 개발자

렌터카 예약 도메인을 기반으로 실제 서비스 운영을 고려한 구조 설계 경험

운영 리스크 제어

가격 정책 설계

데이터 무결성 보장

프로젝트 개요

01

CarPick

렌터카 예약 플랫폼 | Backend Lead | 9인 팀 프로젝트

02

OnBid

공공데이터 기반 공매 물건 조회 서비스 | Solo 프로젝트

03

Python Crawling

Selenium 기반 렌터카 데이터 크롤링 | DB 설계 근거 확보

01

렌터카 예약 플랫폼 CarPick

개발 기간	2025.12.05 ~ 2026.01.29
참여 인원	총 9명 (Backend 4 / Frontend 5)
기여도	Backend Lead (백엔드 전반 설계·구현 주도)
기술 스택	Java, Spring Boot, MariaDB, MyBatis, React

CarPick 핵심 성과

Reservation Snapshot
예약 시점 데이터를 컬럼으로 고정 저장하여 정책 변경과 무관하게 금액 무결성 확보

Server-side Price Validation
서버 단일 가격 엔진에서 모든 가격 계산을 수행하여 클라이언트 기반 위변조 차단

Price Module Architecture
SRP 원칙에 따라 정책·할인·기간 계산을 모듈화하여 영향 범위 최소화

Concurrency Control
Row-Level Pessimistic Lock + 멀티스레드 테스트로 동시 예약 중복 완전 차단

Service Layer Design
SRP 기반 4개 서비스 분리 + 테스트 코드 자동 검증

DB Architecture
핵심 13개 테이블 설계, 확장 시에도 변경 없이 유지

CarPick 추가 성과

Admin & Operational Design

활성/비활성 토글 + Inline Edit로 운영자 직접 정책 제어 구조 확보

Back-end → Service Flow

API 연동·URL 상태 설계를 통해 서비스 전체 흐름을 바라보는 시야 확장

V1 → V2 API 리디자인

마감 2주 전 구조적 한계 인식, 4일 만에 전환 완성

API-First Collaboration

API Contract 선행 정의로 프런트/백엔드 병렬 개발 구조 확보

예약 흐름 기반 데이터 모델 설계

데이터 구조 자체로 비즈니스 정책을 통제할 수 있도록 설계

13
핵심 테이블
초기 설계

→

29
확장 과정
비즈니스 요구 반영

→

28
정규화 최적화
불필요 테이블 제거

설계 원칙

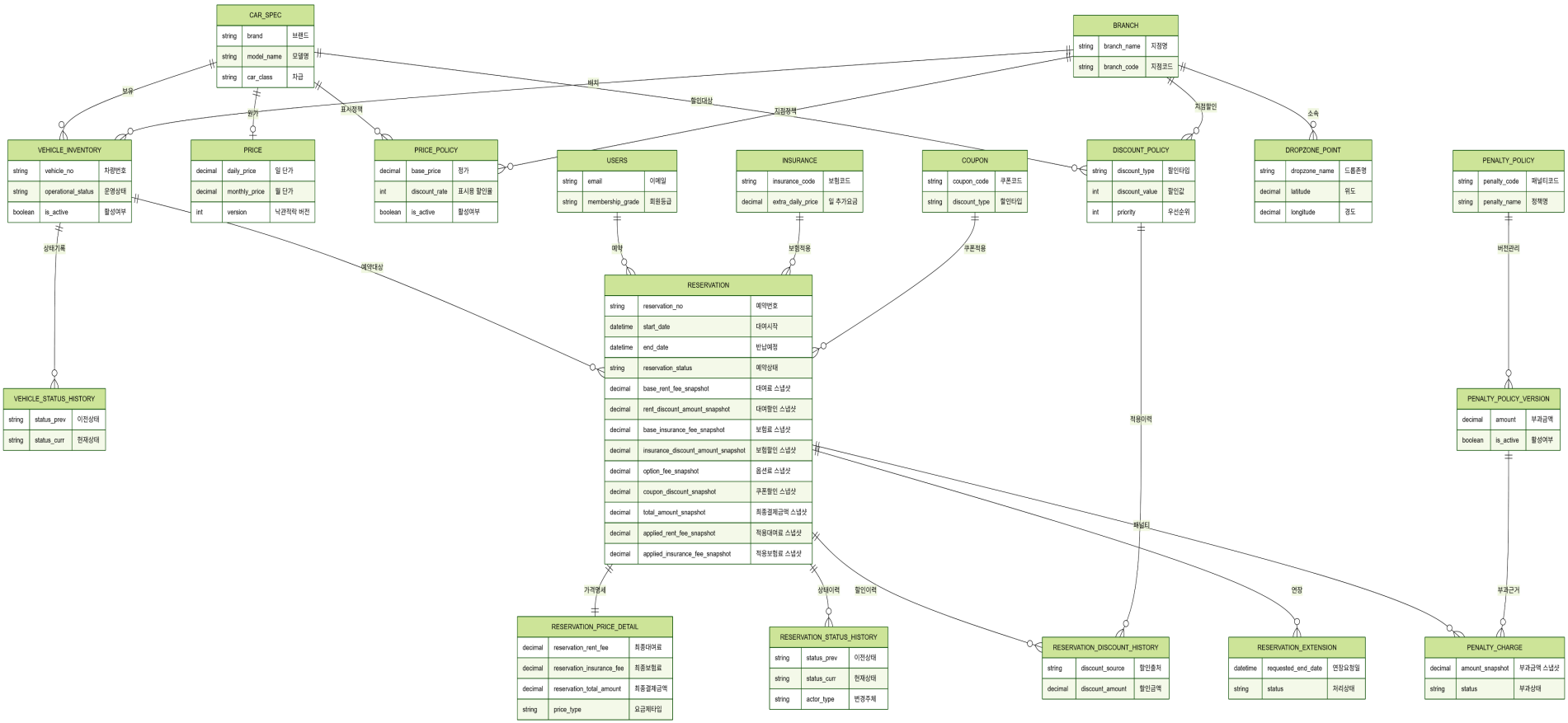
기본은 반정규화로 가되, 데이터가 분명한 의미를 가지면 정규화

적용

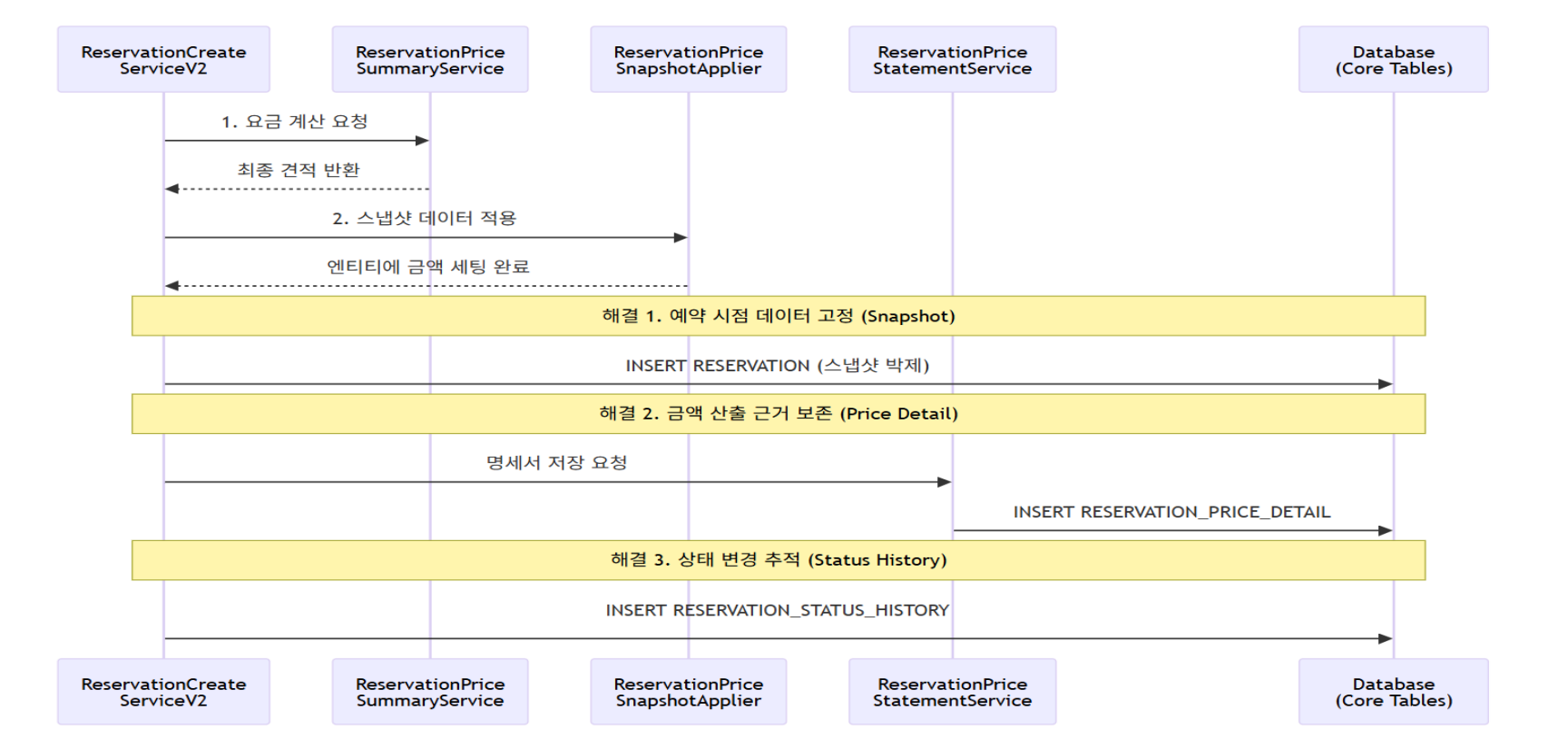
PRICE_POLICY를 데이터 의미 기준으로 정규화 → 지점별/차종별 정책 제어 가능
→ 악성재고·시즌 테이블이 자연스럽게 불필요해져 제거 (29 → 28)

ERD

지점에 속한 **차량 재고**를 기준으로, 기간 동안 **가용 상태**를 확인하고, 해당 조건에 맞는 **가격**을 계산한 뒤, **보험**을 적용하여 최종 **예약**을 생성하는 구조



Reservation Snapshot Architecture



Reservation Snapshot

PROBLEM

- 예약 생성 이후 가격 정책·보험·할인·재고 상태가 변경되면 조회 시점 기준 재계산으로 금액 불일치 발생

SOLUTION

- Snapshot: 예약 시점 금액/조건을 컬럼으로 고정 저장
- Price Detail: 금액 산출 근거를 별도 테이블로 분리
- Status History: 상태 변경 이력 추적 구조 설계

RESULT

- 정책 변경과 무관하게 예약 금액 일관성 유지
- 금액 계산 근거 추적 가능 (정산/분쟁 대응)
- 예약 데이터 신뢰성 확보 및 운영 안정성 향상

Server-side Price Validation

PROBLEM

- 프런트에서 가격 계산 시 브라우저 조작 및 정책 불일치 발생

  → 환불/보상 등 직접적인 매출 손실 위험

SOLUTION

- 서버 단일 가격 엔진에서 모든 가격 계산 수행

  → 프런트에는 결과만 전달하는 구조로 전환

RESULT

- 클라이언트 기반 공격 벡터 제거
- 가격 정책 변경시에도 서버 기준 일관된 계산·검증 가능
- Snapshot과 결합하여 가격 무결성 + 보안 + 재현 가능성 확보

Price Calculation & Policy Architecture

SRP 원칙에 따라 정책·할인·기간 계산을 모듈화하여 정책 변경과 운영 요구에 유연하게 대응

Concurrency Control

비관적 락 | 멀티스레드 테스트

시스템 규모를 고려해 Redis 대신 DB Row-Level Lock을 선택하고,
차량 스펙/재고 분리 설계를 기반으로 재고 Row 단위 동시성 제어를 적용

PROBLEM

- 동일 차량 동시 예약 시 예약·재고 음수 위험

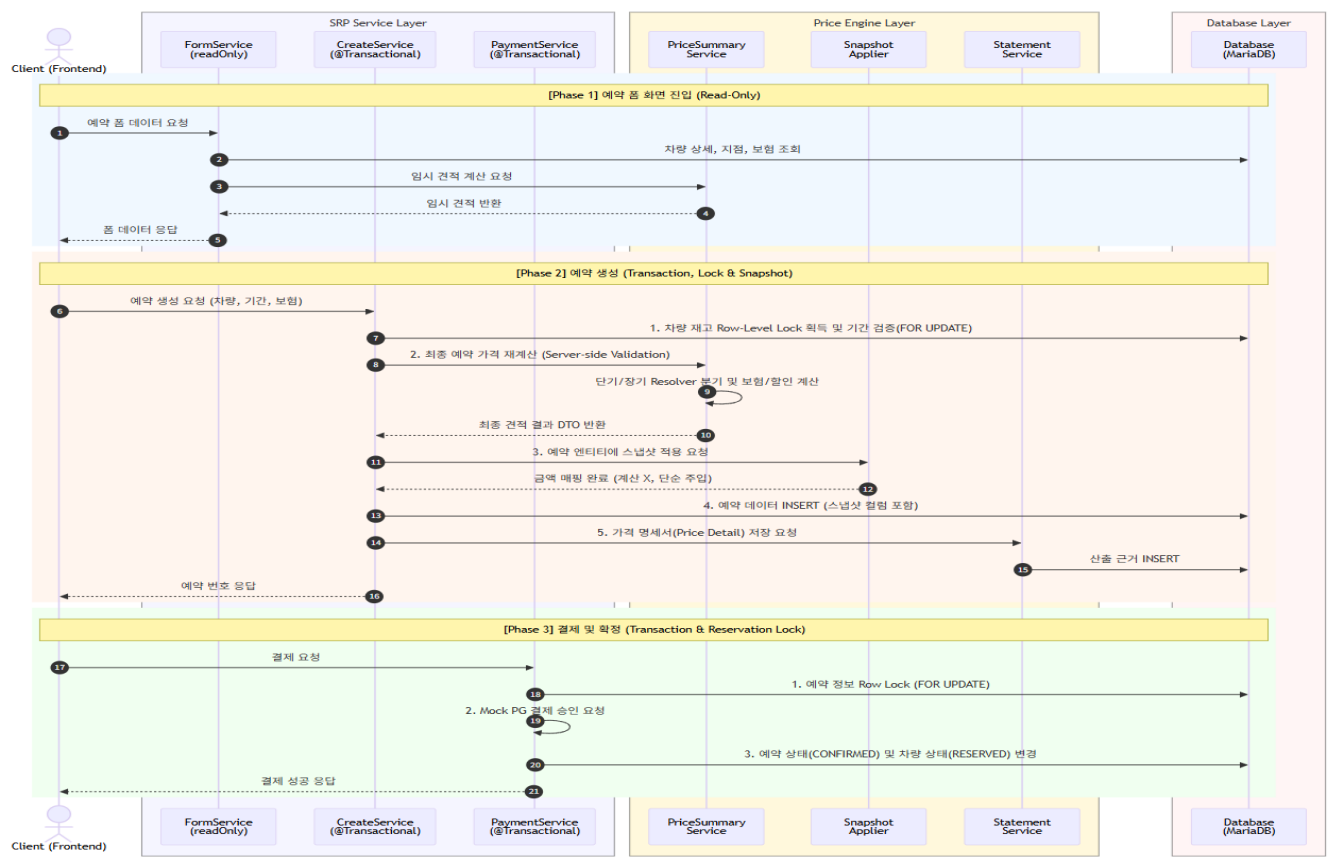
SOLUTION

- Row-Level Pessimistic Lock + @Transactional 조합
- 재고 Row Lock → 기간 겹침 재검증 → 예약 생성 → Commit
- lock 획득 후 기간 겹침 재검증하여 경쟁 상태 방지
- CountDownLatch + ExecutorService 기반 멀티스레드 테스트로 검증

RESULT

- 동시 예약 10건 요청 → 1건 성공 / 9건 실패 확인
- 트랜잭션 종료 시점까지 Lock 유지 → 중복 예약 완전 차단

Reservation Lifecycle Architecture



Service Layer Design

Create 테스트 | Payment 테스트

예약 도메인을 SRP 기반으로 4개 서비스로 분리하고 핵심 비즈니스 로직을 **테스트 코드**로 검증

PROBLEM

- 조회/생성/결제가 단일 서비스에 혼재
  → 트랜잭션 비용 증가 + 유지보수 어려움

SOLUTION

- 역할별 4개 서비스 (**Create / Form / Payment / Read**)로 분리 + 트랜잭션 목적별 최적
- Create 테스트: 예약 생성 + 스냅샷 + 가격 명세 저장까지 검증, DROPZONE 누락 시 예외 처리 검증
- Payment 테스트: 성공 / 멱등 / 거절 / 권한 실패 4가지 시나리오

RESULT

- ReadOnly 서비스와 쓰기 트랜잭션 분리 → DB 커넥션 효율화
- 테스트 코드로 예외 케이스 자동 검증 → 회귀 안정성 확보

API-First Collaboration

프론트 UI 기준으로 설계된 API 응답 구조

PROBLEM

- API 응답 구조(JSON) 변경 → UI 개발이 반복적으로 수정되는 문제 발생
- 화면(UI) 기준으로 API가 설계되지 않으면 필드 누락, 중복 호출, 불필요한 API 추가 요청 발생

SOLUTION

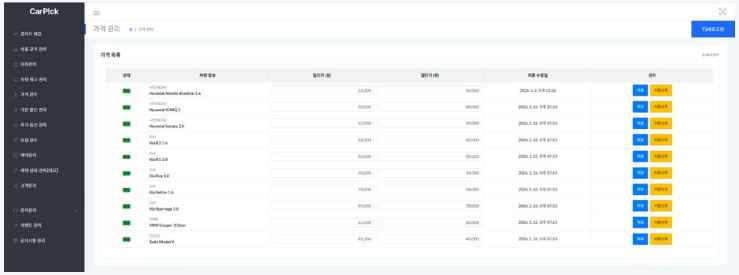
- UI 기준으로 API JSON 구조를 핵심 테이블 13개 확정 전에 먼저 설계
  → API Contract 선 확정 후 개발 진행

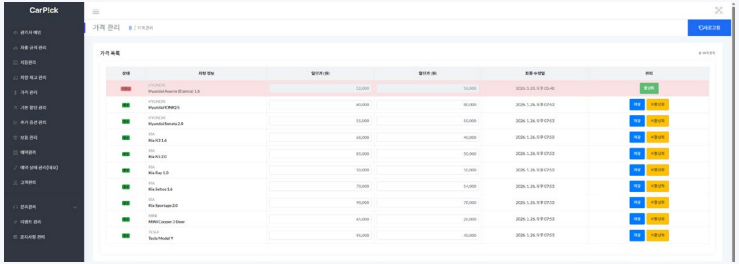
RESULT

- 프론트/백엔드 병렬 개발 가능
  → 커뮤니케이션 비용 감소 + 개발 속도 향상

Admin & Operational Design

관리자 CRUD가 아닌, 운영자가 직접 정책을 제어할 수 있는 운영 친화적 구조 설계





PROBLEM

- 운영 정책 변경 시 개발자가 직접 DB 수정 → 대응 속도 저하
- 지점별/차종별 할인, 리콜 대응 등 화면에서 직접 통제 필요

SOLUTION

- Soft Delete → 활성/비활성 토글 전환
- Inline Edit로 가격 정책 즉시 수정
- Optimistic Lock으로 동시 수정 충돌 방지
- 할인 정책을 지점→차종 단위까지 확장

RESULT

- 개발자 의존 없이 운영자 직접 정책 제어 구조 확보
- 대규모 리콜 시 차종 일괄 비활성화 대응 가능

CarPick 개발일정 (Phase 1, 2, 3차)

핵심 기능을 먼저 구현한 뒤 단계적으로 기능을 확장하는 반복적 개발 방식 (Iterative Development)

Phase 1(12/5 ~ 1/2)

설계 & 확장

- 크롤링 기반 시장 조사로 실제 렌터카 가격·재고 구조 분석
- 핵심 테이블 13개 설계, 최종 구조에서도 안정적으로 유지

구현 & 협업

- 사용자 예약·결제 프로세스 핵심 흐름 (Happy Path) 구현
- 관리자 기능 설계 중심 구현 (Soft Delete / 테이블형 / 인라인 수정)

Phase 2(1/2 ~ 1/11)

설계 & 확장

- 별금 테이블 추가 등 비즈니스 요구에 따른 서비스 확장
- 비관적 락 적용 해서 예약 동시성 확보
- 멀티스레드 테스트 코드로 테스트 완료

구현 & 협업

- UX 관점을 반영하여 API 구조 재정의 (V1 → V2)

Phase 3(1/11 ~ 1/26)

설계 & 확장

- 기존 논리 삭제를 '활성/비활성 토글 버튼' 방식으로 고도화
- UI/UX 개선을 통해 사용자 편의성 및 예약 경험 강화

구현 & 협업

- Dropzone API 연동하여 파일 업로드 기능 구현
- 프론트-백엔드 협업 경험 확보
- 가격 계산 로직을 Factory Pattern 기반 객체지향 구조로 분리 설계

Back-end → Service Flow 설계로 확장

API 단위 개발에서 벗어나 프론트–백엔드 간 데이터 흐름과 사용자 경험을 직접 연결

예약 플로우 관통

예약하기 페이지에 예약 API를 직접 연동하여
검색→예약→결제까지 사용자 흐름을 직접 연결

URL 상태 설계

지점·기간 정보를 Query Parameter로 구조화하여
페이지 간 상태 유지 및 재방문 UX 확보

가격 API 연동

가격 엔진 API를 예약 페이지에 연동하여
프론트–백엔드 간 데이터 흐름을 일관되게 통합

▶ **이 과정에서 기존 API 구조가 사용자 흐름과 맞지 않는 문제를 인식**

사용자 흐름 기반 API 재정의 (V1 → V2)

초기 설계의 구조적 한계를 인식하고 마감 2주 전 API 구조를 전면 재정의

PROBLEM

- 백엔드 중심 설계로 API 구조가 실제 사용자 흐름과 불일치
- 예약하기 가격 서비스가 작동하지 않아 프런트에서 가격 미표시
- 마감 2주 전, "기능은 돌아가지만 구조는 틀린" 상태

SOLUTION

- 사용자 흐름 기반 API 구조 전면 재설계 (V1 → V2)
- 기존 기능을 유지한 상태에서 구조만 단계적으로 전환
- V1 정상 코드는 완전 이식, 비정상 코드만 선별 수정

RESULT

- 4일 만에 V1 → V2 전환 완료 (기능 유지 상태)
- 기능 유지·구조 개선·무결성 강화 동시 달성

02

OnBid

공공데이터 기반 공매 물건 조회 서비스

개발 기간: 2025.11.01 ~ 2025.11.15 (2주) **기술 스택:** Java, Spring Boot, MariaDB, MyBatis, Public API (KAMCO)

한국자산관리공사(KAMCO) 온비드 공공 API를 연동하여 복잡한 공매 데이터를 파싱하고, 사용자 친화적으로 시각화한 웹 서비스

공공데이터 기반 공매 물건 조회 서비스 (OnBid)

온비드 OpenAPI를 활용하여 공매 물건 데이터를 수집하고 외부 데이터를 내부 서비스 데이터로 변환하여 활용

공공 API 연동

온비드 OpenAPI를 WebClient로 호출하여 XML 데이터 수집

데이터 내부화

외부 데이터를 DTO로 변환 후 내부 DB에 저장

조회 서비스 제공

목록 / 상세 / 검색 / 상태별 조회 기능 구현

동기화 기능

수동 동기화 API를 통해 최신 공매 데이터 반영

외부 공공데이터를 서비스 도메인으로 재구성한 설계

외부 공공데이터(XML)를 그대로 사용하지 않고 서비스 운영에 맞는 내부 도메인 구조로 재구성

WebClient 호출 → XML 응답 DOM 파싱 → DTO 변환 → Entity 변환 → DB 저장

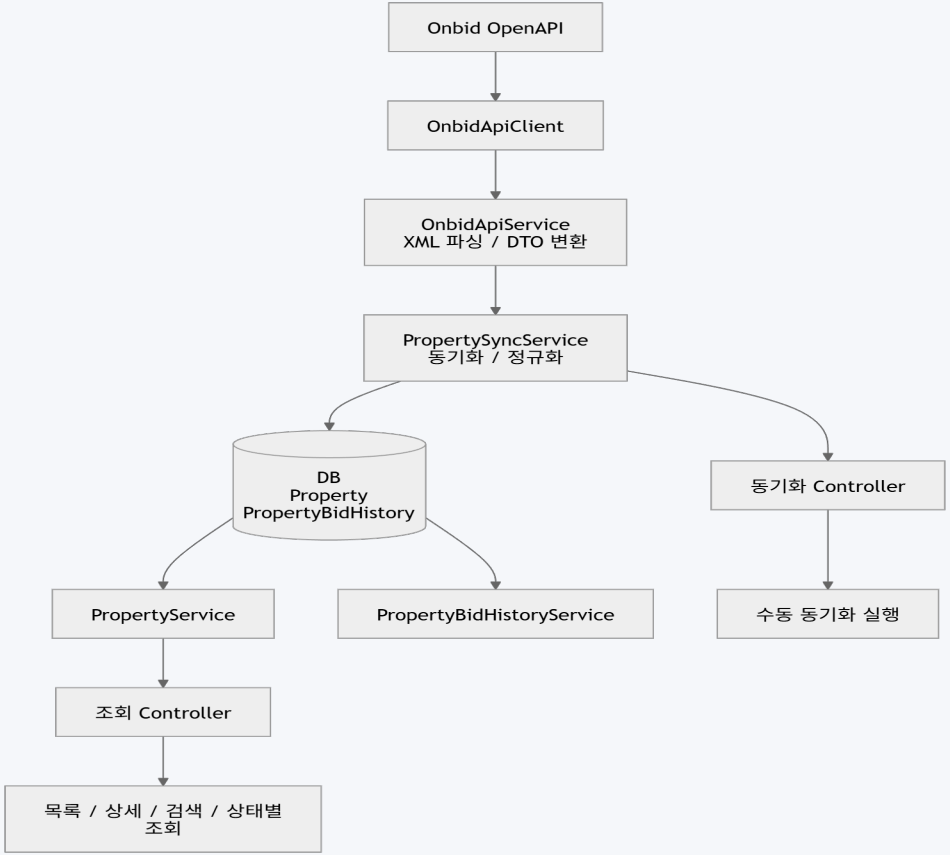
구현

- 물건(Property)과 회차별 입찰 이력(PropertyBidHistory) 2테이블 분리 설계
- 동기화 로직: 신규 insert / 기존 patch / 회차 중복 체크 후 이력 추가
- 조회·검색·필터링·상태별 조회 비즈니스 로직 구현

의미

- 외부 API 데이터를 내부 서비스 모델로 변환하는 데이터 파이프라인 설계 경험
- 이후 Kafka·Redis 도입을 통한 실시간 동기화 확장 예정

OnBid 아키텍처


Onbid OpenAPI
OnbidApiClient
OnbidApiService
XML 파싱 / DTO 변환
PropertySyncService
동기화 / 정규화
DB
Property
PropertyBidHistory
동기화 Controller
PropertyService
PropertyBidHistoryService
수동 동기화 실행
조회 Controller
목록 / 상세 / 검색 / 상태별
조회

03 Python Crawling

Python | 실데이터

렌터카 도메인 설계를 위해 실데이터가 필요했고,
경험이 없던 Python을 빠르게 학습하여 Selenium 기반 크롤러를 구현하고 데이터를 확보

PROBLEM

- CarPick DB 설계를 위해 실제 렌터카 가격·차종·재고 구조를 데이터로 분석할 필요
- 추측이 아닌 실데이터 기반 설계를 목표
- Python 경험이 없었지만 하루 학습 후 Selenium 기반 크롤러 구현

SOLUTION

- Python + Selenium을 활용해 렌터카 사이트 차량 목록 크롤링
- Lazy Loading 대응 스크롤 크롤링
- 지점별 가격, 차종, 렌트 조건 데이터 수집 및 구조화

RESULT

- 수집한 실데이터를 분석하여 CarPick 예약·가격·재고 중심 Core Table 13개 설계의 근거로 확보
- 새로운 기술을 빠르게 학습하고 실제 문제 해결에 적용한 경험